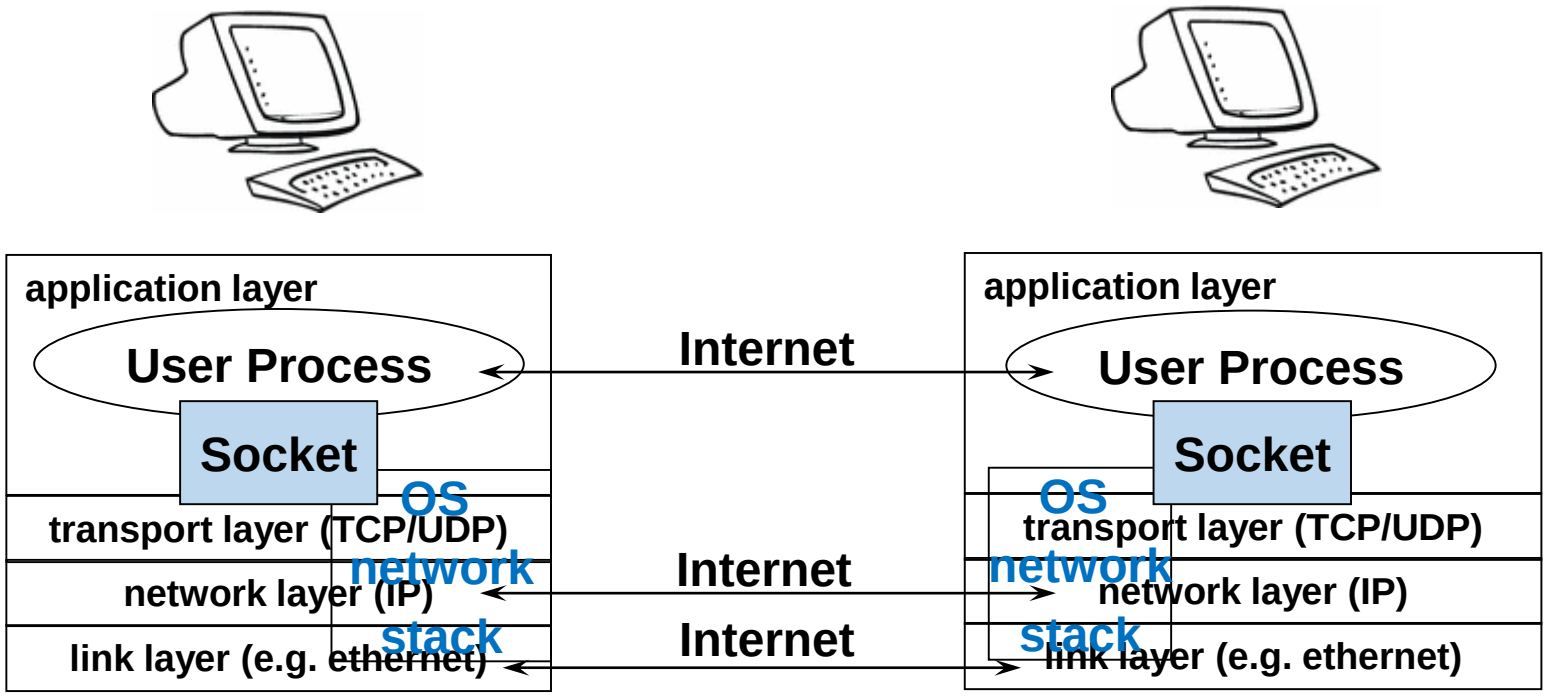
“Socket Programming”
Computer Networks
Lab-6

Socket and Process Communication



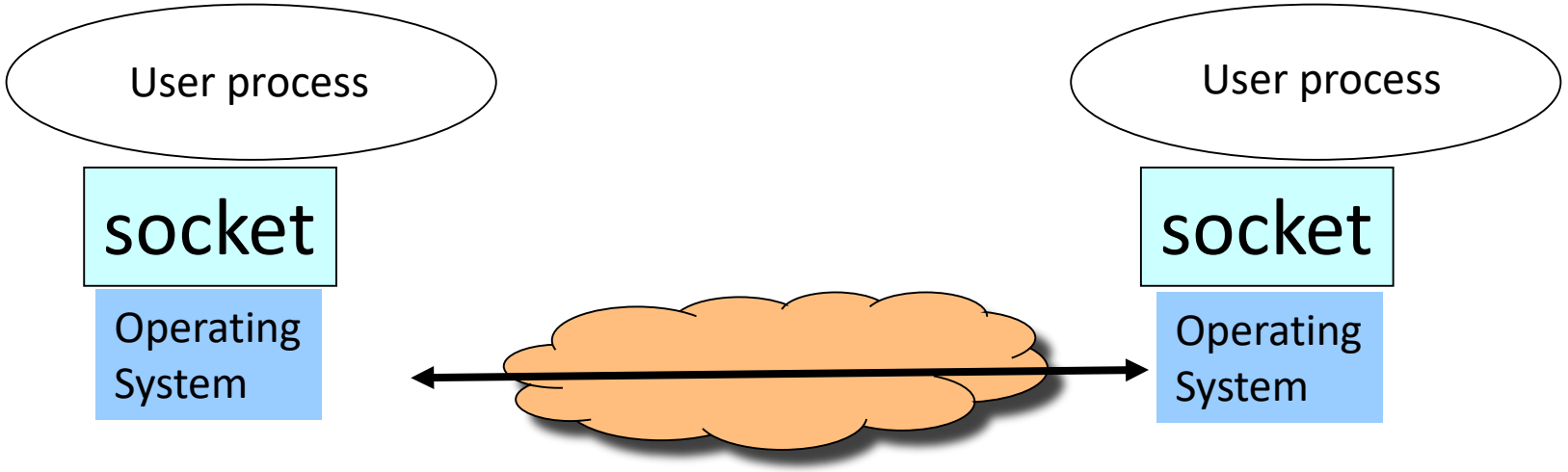
The interface that the OS provides to its networking subsystem

Delivering the Data: Division of Labor

- Network
  - Deliver data packet to the destination host
  - Based on the destination IP address
- Operating system
  - Deliver data to the destination socket
  - Based on the destination port number (e.g., 80)
- Application
  - Read data from and write data to the socket
  - Interpret the data (e.g., render a Web page)

Socket: End Point of Communication

- Sending message from one process to another
  - Message must traverse the underlying network
- Process sends and receives through a "socket"
  - In essence, the doorway leading in/out of the house
- Socket as an Application Programming Interface
  - Supports the creation of network applications



Socket

- A way to speak to other programs using standard Unix file descriptors.
- Application layer protocols rely on socket to create connection
- Low level end-point to process information between client and server machines

Mainly Two-types of Sockets

1. Stream Sockets: referred to as ‘SOCK_STREAM’
   - reliable two-way connected communication streams. If one outputs two items into the socket in the order "1, 2", they will arrive in the order "1, 2" at the opposite end.
   - Uses TCP protocol
   - Example: HTTP, FTP, etc.
2. Datagram Sockets: referred to as ‘SOCK_DGRAM’
   - connectionless sockets
   - Uses UDP Protocol
   - Example: multiplayer games, streaming audio, video conferencing, etc.
   - Advantage: speed

User Datagram Protocol (UDP):Datagram Socket

UDP	Postal Mail
• Single socket to receive messages	• Single mailbox to receive letters
• No guarantee of delivery	• Unreliable
• Not necessarily in-order delivery	• Not necessarily in-order delivery
• Datagram – independent packets	• Letters sent independently
• Must address each packet	• Must address each mail

Example UDP applications
Multimedia, voice over IP (Skype)

Transmission Control Protocol (TCP):Stream Socket

TCP

- Reliable – guarantee delivery
- Byte stream – in-order delivery
- Connection-oriented – single socket per connection
- Setup connection followed by data transfer

Telephone Call

- Guaranteed delivery
- In-order delivery
- Connection-oriented
- Setup connection followed by conversation

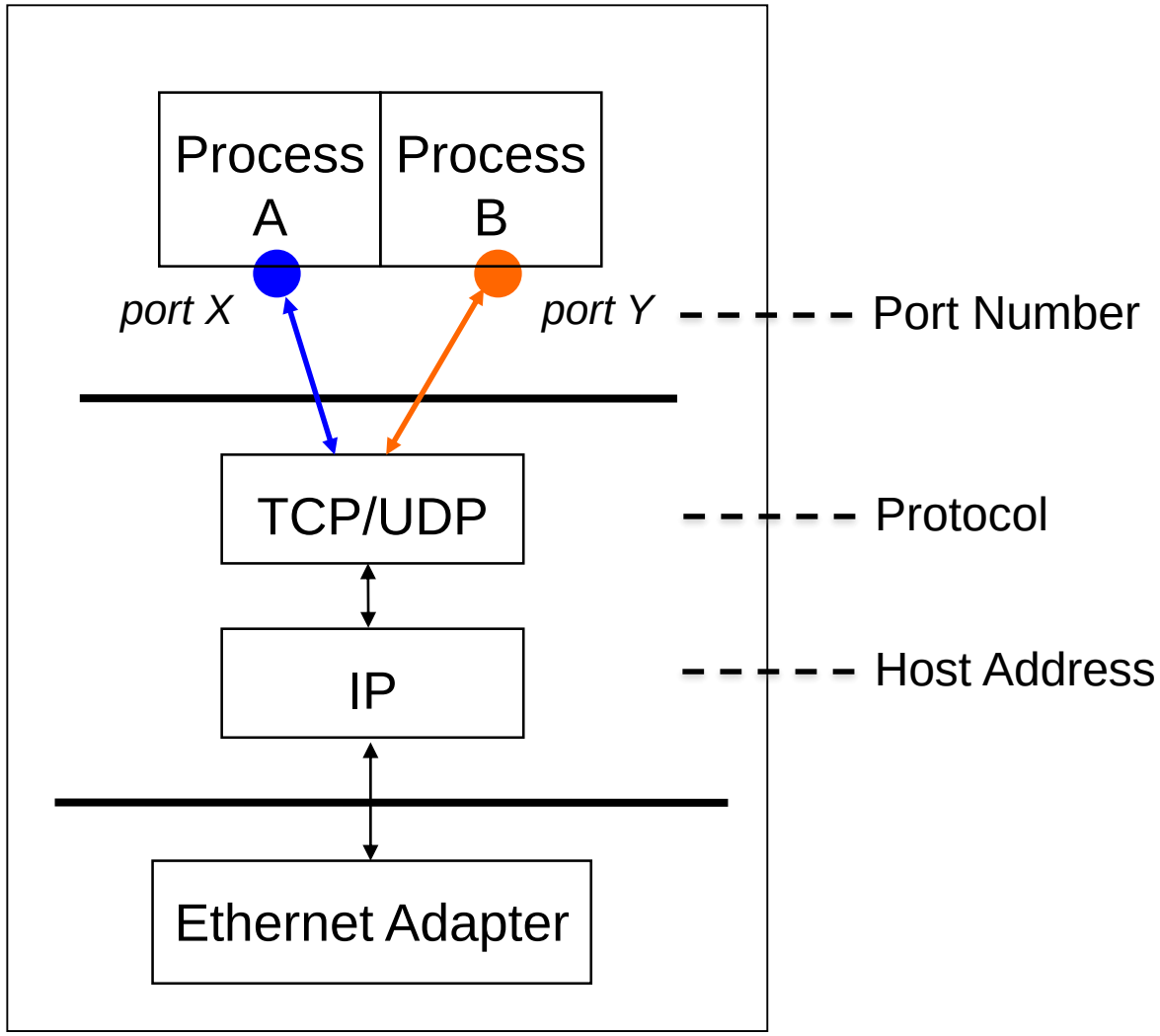
Example TCP applications

Web, Email, Telnet

Socket Identification

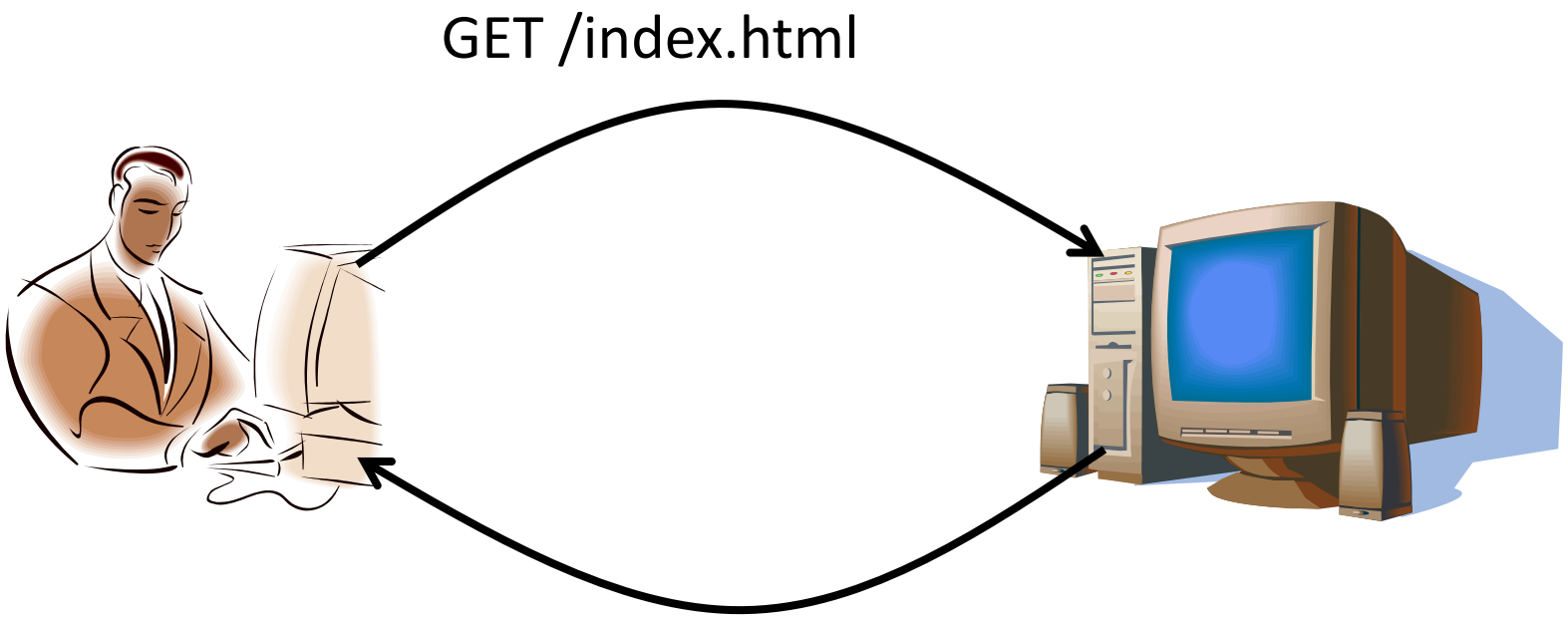
- Communication Protocol
  - TCP (Stream Socket): streaming, reliable
  - UDP (Datagram Socket): packets, best effort
- Receiving host
  - Destination **address** that uniquely identifies the host
  - An **IP address** is a 32-bit quantity
- Receiving socket
  - Host may be running many different processes
  - Destination **port** that uniquely identifies the socket
  - A **port number** is a 16-bit quantity

Socket Identification (Cont.)



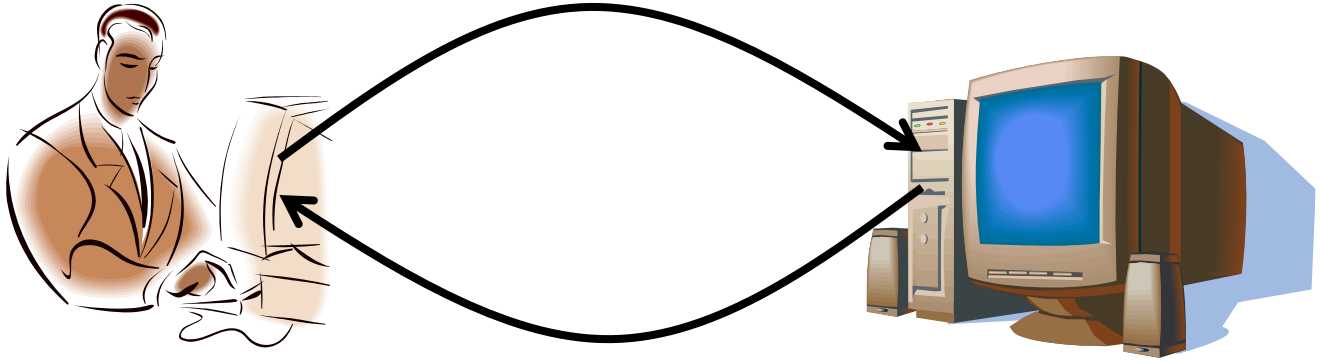
Clients and Servers

- Client program
  - Running on end host
  - Requests service
  - E.g., Web browser
- Server program
  - Running on end host
  - Provides service
  - E.g., Web server



Client-Server Communication

- Client "sometimes on"
  - Initiates a request to the server when interested
  - E.g., Web browser on your laptop or cell phone
  - Doesn't communicate directly with other clients
  - Needs to know server's address
- Server is "always on"
  - Handles services requests from many client hosts
  - E.g., Web server for the www.cnn.com Web site
  - Doesn't initiate contact with the clients
  - Needs fixed, known address



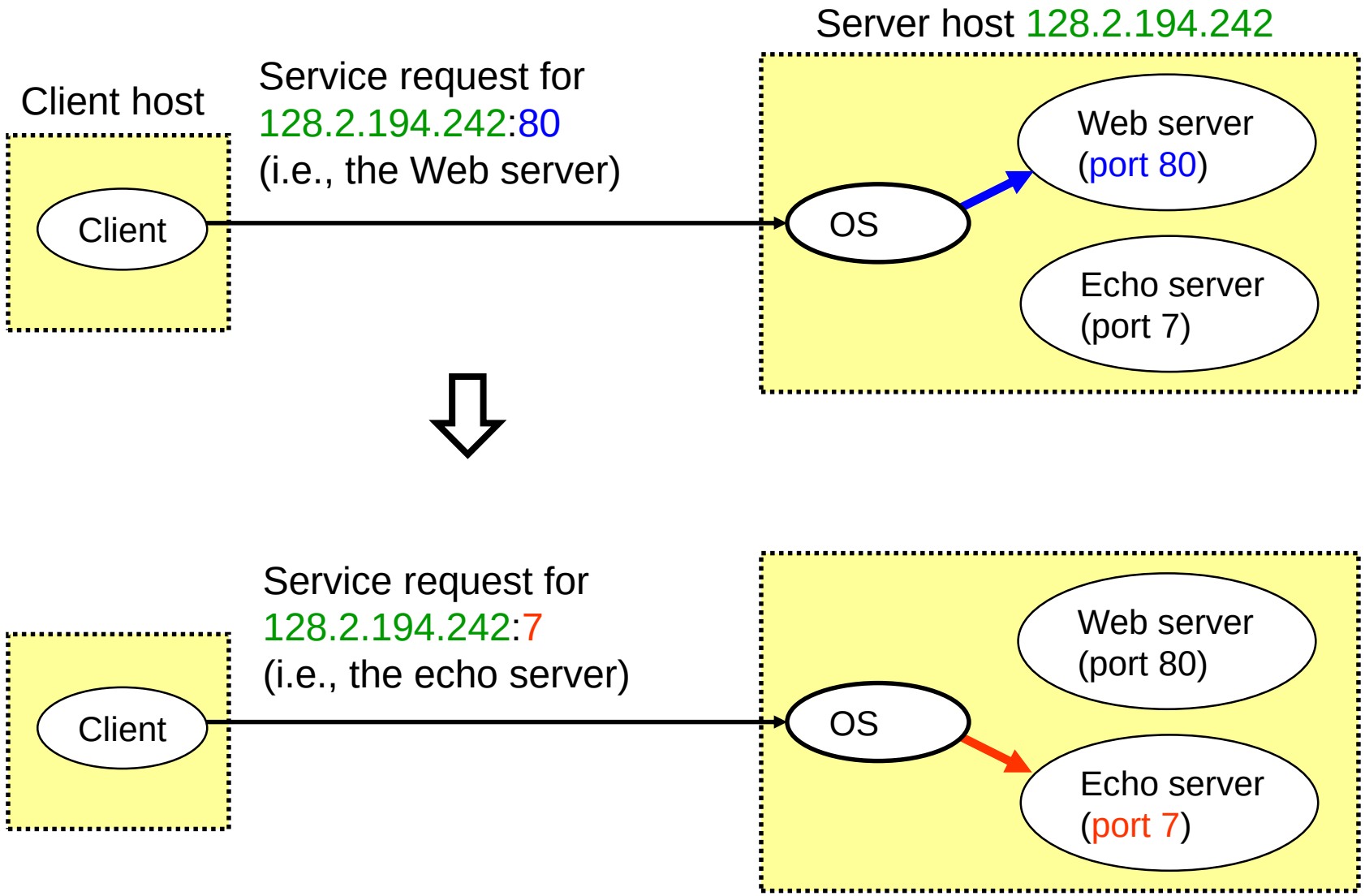
Client and Server Processes

- Client process
  - process that initiates communication
- Server Process
  - process that waits to be contacted

Knowing What Port Number To Use

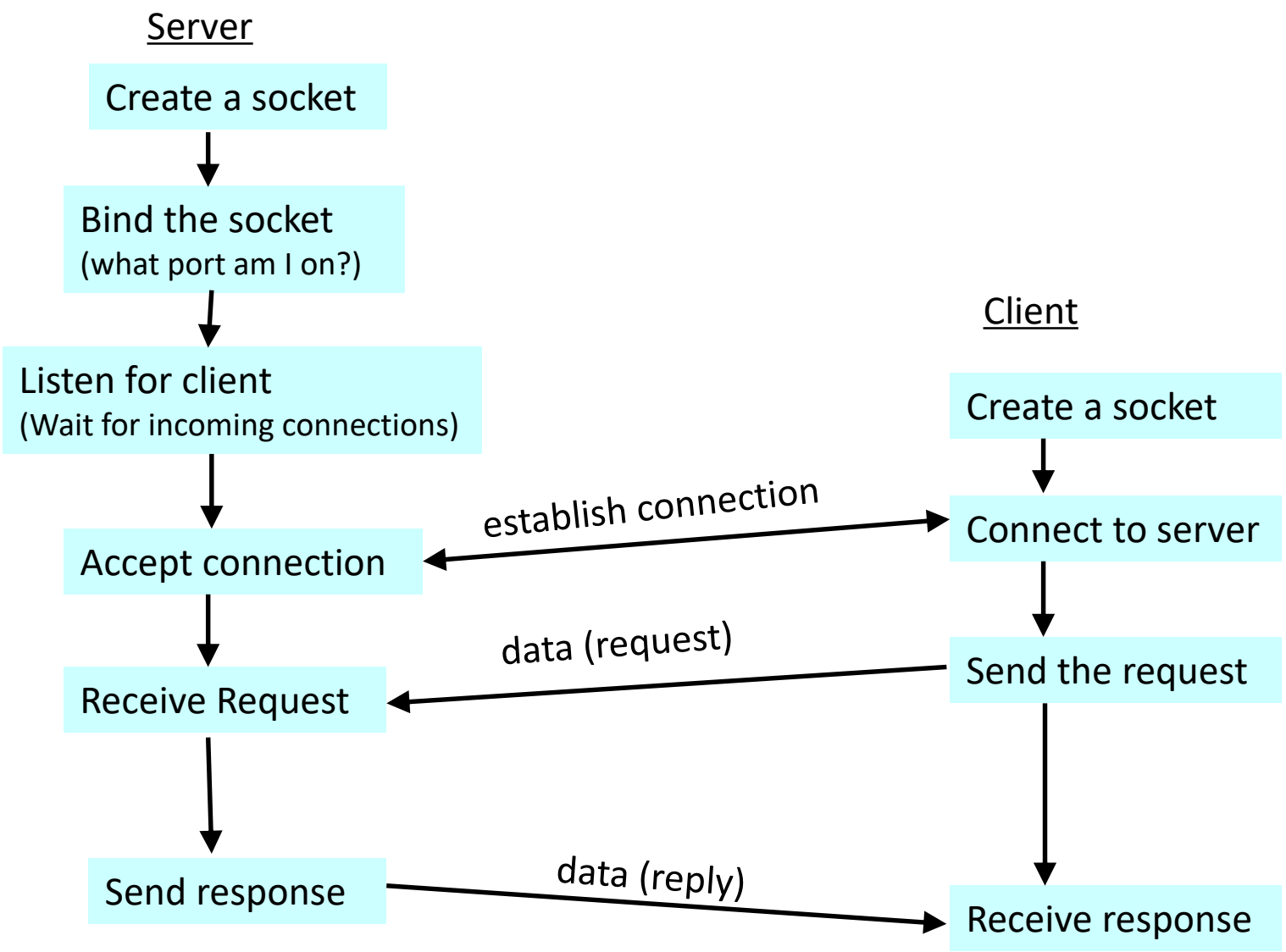
- Popular applications have well-known ports
  - E.g., port 80 for Web and port 25 for e-mail
  - See http://www.iana.org/assignments/port-numbers
- Well-known vs. ephemeral ports
  - Server has a well-known port (e.g., port 80)
    - Between 0 and 1023 (requires root to use)
  - Client picks an unused ephemeral (i.e., temporary) port
    - Between 1024 and 65535
- Uniquely identifying traffic between the hosts
  - Two IP addresses and two port numbers
  - Underlying transport protocol (e.g., TCP or UDP)

Using Ports to Identify Services



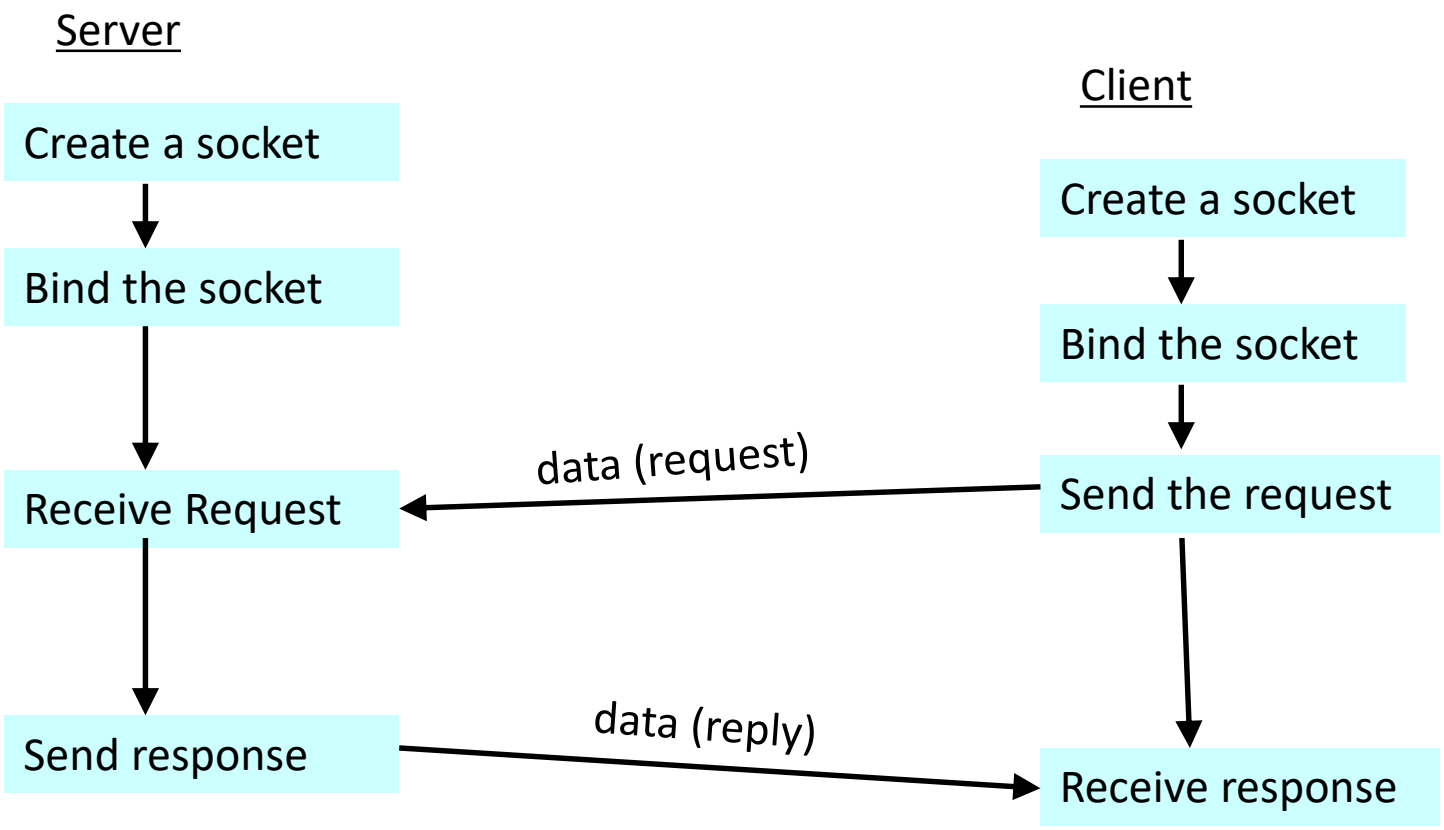
Client-Server Communication

Stream Sockets (TCP): Connection-oriented



Client-Server Communication

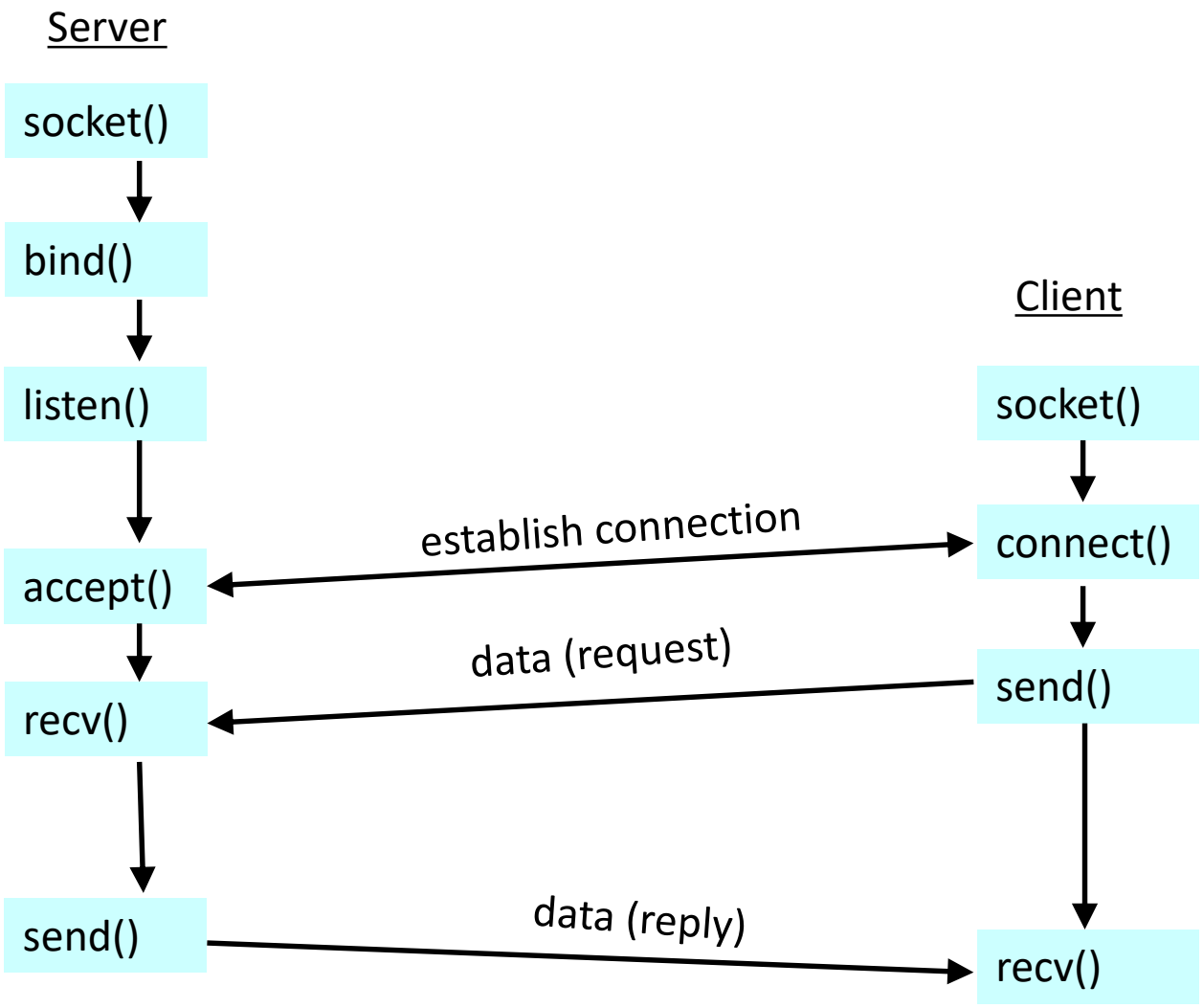
Datagram Sockets (UDP): Connectionless



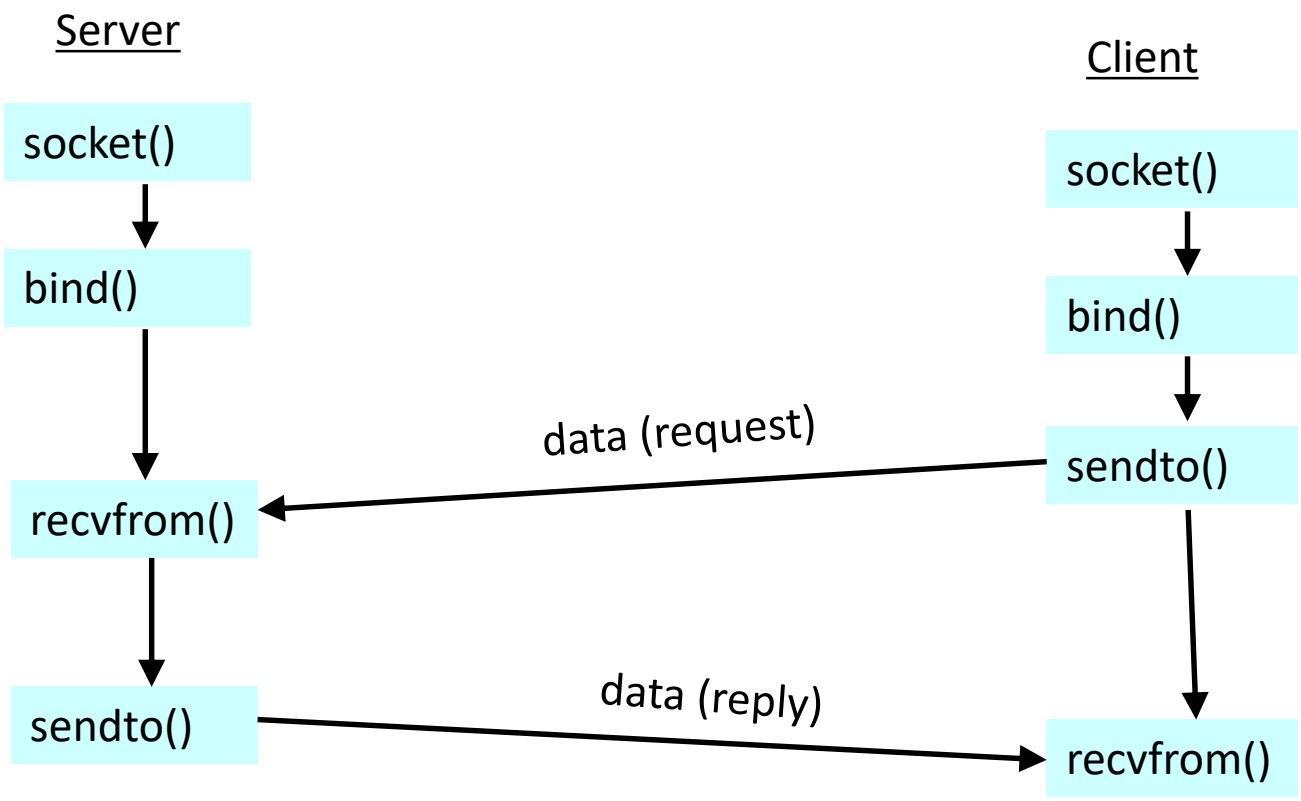
UNIX Socket API

- Socket interface
  - Originally provided in Berkeley UNIX
  - Later adopted by all popular operating systems
  - Simplifies porting applications to different OSes
- In UNIX, everything is like a file
  - All input is like reading a file
  - All output is like writing a file
  - File is represented by an integer file descriptor
- API implemented as system calls
  - E.g., connect, send, recv, close, …

Connection-oriented Example (Stream Sockets -TCP)



Connectionless Example
(Datagram Sockets - UDP)



Client: Learning Server Address/Port

- Server typically known by name and service
  - E.g., "www.cnn.com" and "http"
- Need to translate into IP address and port #
  - E.g., "64.236.16.20" and "80"

- Get address info with given host name and service
  -
```
int getaddrinfo(  char *node,
                  char *service
                  struct addrinfo *hints,
                  struct addrinfo **result)
```

  - *node: host name (e.g., "www.cnn.com") or IP address
  - *service: port number or service listed in */etc/services* (e.g. ftp)
  - hints: points to a *struct addrinfo* with known information

Client: Learning Server Address/Port (cont.)

- Data structure to host address information

```
struct addrinfo {
            int                                 ai_flags;

            int                                 ai_family;
    //e.g. AF_INET for IPv4
            int                                 ai_socketype;
    //e.g. SOCK_STREAM for TCP
            int                                 ai_protocol;
    //e.g. IPPROTO_TCP
            size_t                      ai_addrlen;
            char                        *ai_canonname;
            struct sockaddr      *ai_addr;      // point to
sockaddr struct
            struct addrinfo      *ai_next;
}
```

- Example

```
hints.ai_family = AF_UNSPEC;     // don't care IPv4 or IPv6
hints.ai_socktype = SOCK_STREAM; // TCP stream sockets
int status = getaddrinfo("www.cnn.com", "80", &hints, &result);
// result now points to a linked list of 1 or more addrinfos
// etc.
```

Client: Creating a Socket

- Creating a socket
  - `int socket(int domain, int type, int protocol)`
  - Returns a file descriptor (or handle) for the socket
- Domain: protocol family
  - PF_INET for IPv4
  - PF_INET6 for IPv6
- Type: semantics of the communication
  - SOCK_STREAM: reliable byte stream (TCP)
  - SOCK_DGRAM: message-oriented service (UDP)
- Protocol: specific protocol
  - UNSPEC: unspecified
  - (PF_INET and SOCK_STREAM already implies TCP)
- Example

```
sockfd = socket(result->ai_family,
                result->ai_socktype,
                result->ai_protocol);
```

Client: Connecting Socket to the Server

- Client contacts the server to establish connection
  - Associate the socket with the server address/port
  - Acquire a local port number (assigned by the OS)
  - Request connection to server, who hopefully accepts
  - connect is **<u>blocking</u>**
- Establishing the connection
  - `int connect(int sockfd,struct sockaddr *server_address, socketlen_t addrlen)`
  - Arguments: socket descriptor, server address, and address size
  - Returns 0 on success, and -1 if an error occurs
  - E.g. `connect(sockfd, result->ai_addr, result->ai_addrlen);`

Client: Sending Data

- Sending data
  - `int send(int sockfd, void *msg, size_t len, int flags)`
  - Arguments: socket descriptor, pointer to buffer of data to send, and length of the buffer
  - Returns the number of bytes written, and -1 on error
  - send is **<u>blocking</u>**: return only after data is sent
  - Write short messages into a buffer and send once

Client: Receiving Data

- Receiving data
  - `int recv(int sockfd, void *buf, size_t len, int flags)`
  - Arguments: socket descriptor, pointer to buffer to place the data, size of the buffer
  - Returns the number of characters read (where 0 implies "end of file"), and -1 on error
  - Why do you need len? What happens if buf's size < len?
  - recv is **<u>blocking</u>**: return only after data is received

Server: Server Preparing its Socket

- Server creates a socket and binds address/port
  - Server creates a socket, just like the client does
  - Server associates the socket with the port number

- Create a socket
  - `int socket(int domain, int type, int protocol)`

- Bind socket to the local address and port number
  - `int bind(int sockfd, struct sockaddr *my_addr, socklen_t addrlen)`

Server: Allowing Clients to Wait

- Many client requests may arrive
  - Server cannot handle them all at the same time
  - Server could reject the requests, or let them wait
- Define how many connections can be pending
  - `int listen(int sockfd, int backlog)`
  - Arguments: socket descriptor and acceptable backlog
  - Returns a 0 on success, and -1 on error
  - Listen is **non-blocking**: returns immediately
- What if too many clients arrive?
  - Some requests don't get through
  - The Internet makes no promises…
  - And the client can always try again

Server: Accepting Client Connection

- Now all the server can do is wait…
  - Waits for connection request to arrive
  - **<u>Blocking</u>** until the request arrives
  - And then accepting the new request

- Accept a new connection from a client
  - `int accept(int sockfd, struct sockaddr *addr,socketlen_t *addrlen)`
  - Arguments: sockfd, structure that will provide client address and port, and length of the structure
  - Returns descriptor of socket for this new connection

Client and Server: Cleaning House

- Once the connection is open
  - Both sides and read and write
  - Two unidirectional streams of data
  - In practice, client writes first, and server reads
  - … then server writes, and client reads, and so on
- Closing down the connection
  - Either side can close the connection
  - … using the `int close(int sockfd)`
- What about the data still "in flight"
  - Data in flight still reaches the other end
  - So, server can `close()` before client finishes reading

Server: One Request at a Time?

- Serializing requests is inefficient
  - Server can process just one request at a time
  - All other clients must wait until previous one is done
  - What makes this inefficient?
- May need to time share the server machine
  - Alternate between servicing different requests
    - Do a little work on one request, then switch when you are waiting for some other resource (e.g., reading file from disk)
    - "Nonblocking I/O"
  - Or, use a different process/thread for each request
    - Allow OS to share the CPU(s) across processes
  - Or, some hybrid of these two approaches

See Real Clients and Servers

- Apache Web server
  - Open source server first released in 1995
  - Name derives from “a patchy server” ;-)
  - Software available online at http://www.apache.org
- Mozilla Web browser
  - http://www.mozilla.org/developer/
- Sendmail
  - http://www.sendmail.org/
- BIND Domain Name System
  - Client resolver and DNS server
  - http://www.isc.org/index.pl?/sw/bind/
- …

Client-socket

Important functions

- Call to socket(): To create Socket
  - Need to specify the type of socket (UDP or TCP)
- Call to connect(): To establish connection with a remote address
  - Need to specify the IP address and port number
- Call send() or recv(): To send or receive data from other socket

Client-socket

Sequence of function calls

socket()

↓

connect()

↓

recv()/send()

Server-socket

Important functions

- Call to socket(): To create Socket
  - Need to specify the type of socket (UDP or TCP)
- Call to bind(): To bind the socket with an IP address and port where it listens for connection
- Call to accept(): To accept a connection
- Call send() or recv(): To send or receive data from other socket

Server-socket

Sequence of function calls

socket()

↓

bind()

↓

listen()

↓

accept()

↓

send()/recv()

Example: tcp_client.c

```
#include<stdio.h>
#include<stdlib.h>

#include<sys/types.h>
#include<sys/socket.h>
#include<netinet/in.h>
#include <unistd.h>

int main()
{

 int network_socket;
 int connection_status;
 char server_data[300];

// create a socket by calling function socket()
 network_socket = socket(AF_INET, SOCK_STREAM, 0);

//this socket will connect to another socket from other side
//call the function connect() for connecting with other socket
//before connection we need to specify the address to connect with
//INET header helps here

//Specifying address for the socket

struct sockaddr_in server_address;
server_address.sin_family = AF_INET;
server_address.sin_port = htons(9002);
server_address.sin_addr.s_addr = INADDR_ANY;
```

tcp_client.c contd…

```
//after creating the address call the function connect()

connection_status = connect(network_socket,(struct sockaddr *) &server_address, sizeof(server_address));

if(connection_status == -1)
{
  printf("Error in creating the connection!\n");
}

recv(network_socket, &server_data, sizeof(server_data),0);
//print the received data
printf("The data received from server is: %s \n", server_data);
//closing the socket
close(network_socket);

return 0;
}
```

```
sweta@sweta-VirtualBox:~/networks$ make tcp_client
cc      tcp_client.c   -o tcp_client
```

Example: tcp_server.c

```
#include<stdio.h>
#include<stdlib.h>

#include<sys/types.h>
#include<sys/socket.h>
#include<netinet/in.h>
#include <unistd.h>

int main()
{
 char server_msg[300]="Hello Client! We are connected now!!!";
 //creating socket
 int server_socket;
     server_socket = socket(AF_INET, SOCK_STREAM, 0);

  //defining server address
  struct sockaddr_in server_address;
  server_address.sin_family = AF_INET;
  server_address.sin_port = htons(9002);
  server_address.sin_addr.s_addr = INADDR_ANY;
```

tcp_server.c contd…

```
  // binding the socket to specific IP address and port number
   bind(server_socket, (struct sockaddr *) &server_address, sizeof(server_address));

   listen(server_socket, 5);

   int client_socket;
   client_socket = accept(server_socket, NULL, NULL);

   //send message to client
   send(client_socket, server_msg, sizeof(server_msg),0);

   //close the server socket
   close(server_socket);

return 0;
}
```

```
sweta-VirtualBox:~/networks$ make tcp_server
 tcp_server.c    -o tcp_server
```

Run the Compiled Codes

- Open two different terminals, one for server program and another for client program
- Run the server program first

```
sweta-VirtualBox:~/networks$ ./tcp_server
sweta-VirtualBox:~/networks$
```

- Next run the client program (following output is corresponding the example code)

```
sweta@sweta-VirtualBox:~/networks$ ./tcp_client
The data received from server is: Hello Client! We are conne
cted now!!!
sweta@sweta-VirtualBox:~/networks$
```

References

- Beej's guide to network programming
- https://www.cs.princeton.edu/courses/archive/spring12/cos461/syllabus.html
- https://www.youtube.com/watch?v=LtXEMwSG5-8